

Chapter 05

Processes Scheduling



Md Mahmudur Rahman
Assistant Professor, IIT
mahmud@juniv.edu

Process Scheduling

- Process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.
- Process scheduling is an essential part of a Multiprogramming operating systems.
- Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using **time multiplexing**.

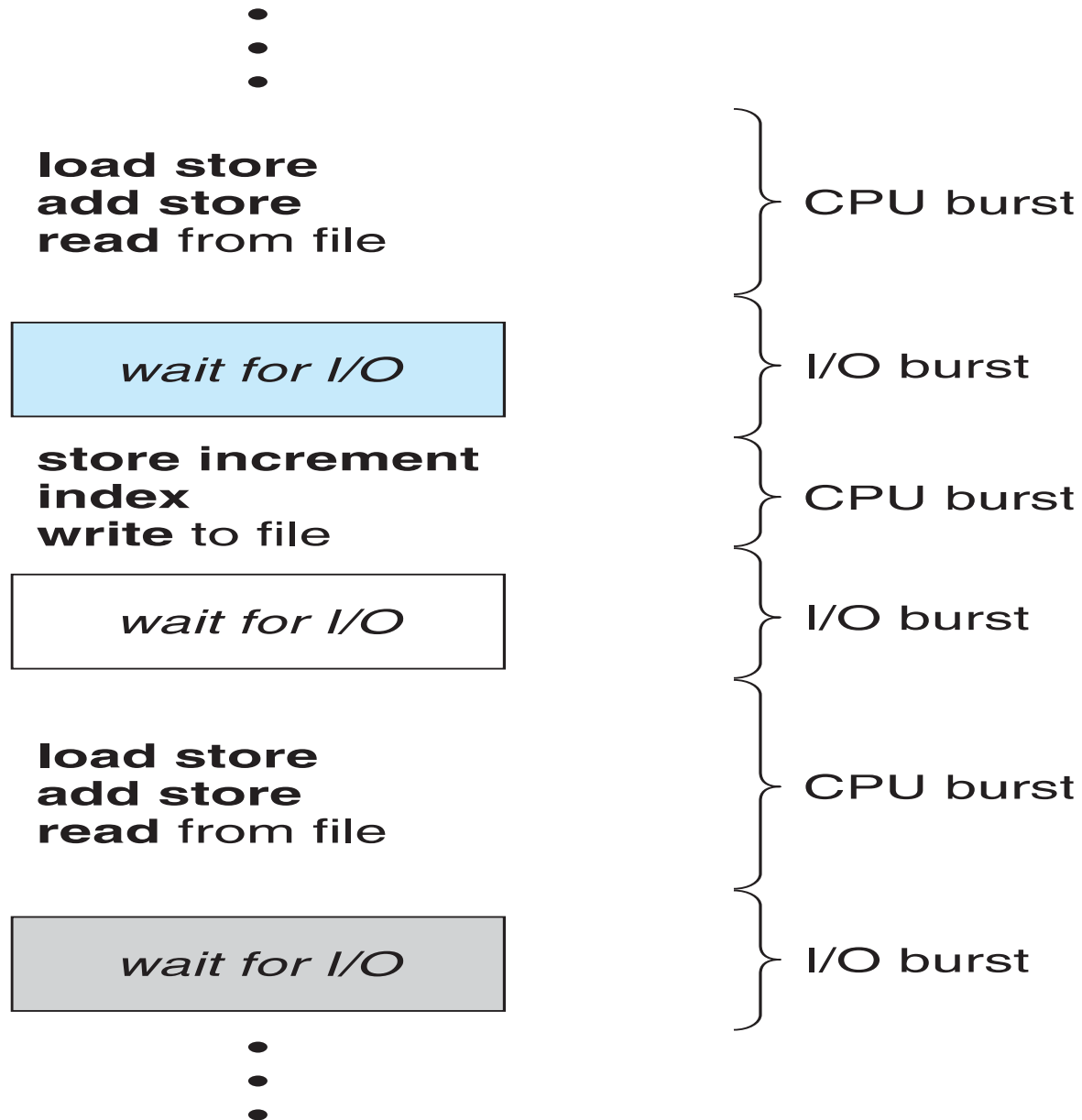
Basic Concepts

- In a system with a single CPU core, only one process can run at a time. Others must wait until the CPU's core is free and can be rescheduled. The objective of **multiprogramming** is to have some process always running, to maximize CPU utilization.
- The idea is relatively simple. A process is executed until it must wait, typically for the completion of some I/O request. In a simple computer system, the CPU then just sits idle. All this waiting time is wasted; no useful work is accomplished.
- With multiprogramming, we try to use this time productively. Several processes are kept in memory at one time. When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process. This pattern continues.
- Every time one process has to wait, another process can take over use of the CPU. On a multicore system, this concept of keeping the CPU busy is extended to all processing cores on the system.

CPU-I/O Burst Cycle

- The success of CPU scheduling depends on an observed property of processes: process execution consists of a cycle of CPU execution and I/O wait. Processes alternate between these two states. Process execution begins with a CPU burst. That is followed by an I/O burst, which is followed by another CPU burst, then another I/O burst, and so on. Eventually, the final CPU burst ends with a system request to terminate execution
- The durations of CPU bursts have been measured extensively. Although they vary greatly from process to process and from computer to computer. An I/O-bound program typically has many short CPU bursts. A CPU-bound program might have a few long CPU bursts. This distribution can be important when implementing a CPU-scheduling algorithm.

Basic Concepts...



Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – Number of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

CPU Scheduler

- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates
- Scheduling under 1 and 4 is **non-preemptive**
- All other scheduling is **preemptive**
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

Preemptive Vs Non-preemptive Scheduling

- **Non-preemptive Scheduling:** A scheduling discipline is non-preemptive if, once a process has been given the CPU, the CPU cannot be taken away from that process.
- Following are some characteristics of non-preemptive scheduling:
 1. In non-preemptive system, short jobs are made to wait by longer jobs but the overall treatment of all processes is fair.
 2. In non-preemptive system, response times are more predictable because incoming high priority jobs can not displace waiting jobs.
 3. In non-preemptive scheduling, a scheduler executes jobs in the following two situations.:
 - When a process switches from running state to the waiting state.
 - When a process terminates.

Preemptive Vs Non-preemptive Scheduling

- **Preemptive Scheduling:** A scheduling discipline is preemptive if, once a process has been given the CPU can taken away.
- The strategy of allowing processes that are logically runnable to be temporarily suspended is called Preemptive Scheduling and it is contrast to the "run to completion" method.

Some Scheduling Algorithms

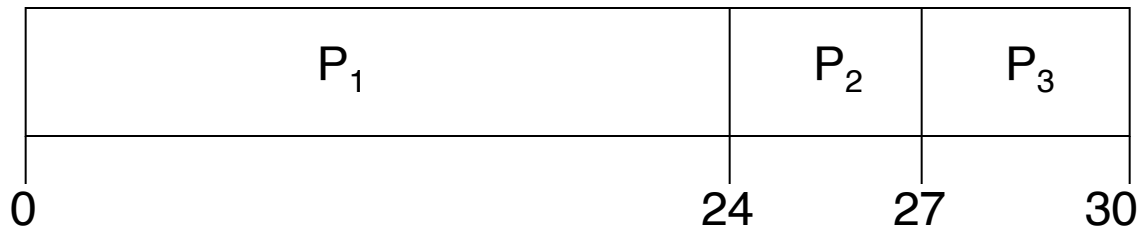
- First-Come, First-Served (FCFS)
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- Round Robin (RR)
- Multiple-Level Queues Scheduling

FCFS

■ Example:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
■ Average waiting time: $(0 + 24 + 27)/3 = 17$

Continue...

Suppose that the processes arrive in the order

$$P_2, P_3, P_1.$$

- The Gantt chart for the schedule is:

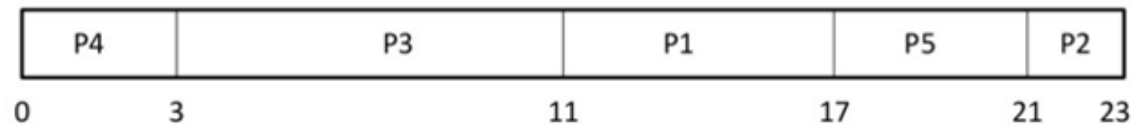


- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case.

Continue...

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

Continue...



Waiting time = Start time - Arrival time

$$P4 = 0 - 0 = 0$$

$$P3 = 3 - 1 = 2$$

$$P1 = 11 - 2 = 9$$

$$P5 = 17 - 4 = 13$$

$$P2 = 21 - 5 = 16$$

Average Waiting Time

$$\frac{0+2+9+13+16}{5}$$

Performance of FCFS Scheduling

- FCFS scheduling is non-preemptive.
- Convoy effect: all the processes wait for the one big process to get off the CPU.
- The FCFS algorithm is particularly troublesome for time-sharing system, where it is important that each user get a share of CPU at regular interval

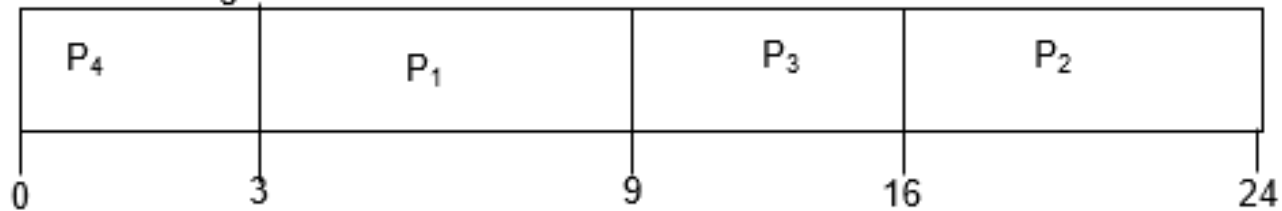
SJF Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time.
- Two schemes:
 - Non-preemptive – once CPU given to the process it cannot be preempted until completes its CPU burst.
 - Preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is known as the Shortest-Remaining-Time-First (SRTF).
- SJF is optimal – gives minimum average waiting time for a given set of processes.

SJF without arrival Time

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

■ SJF scheduling chart

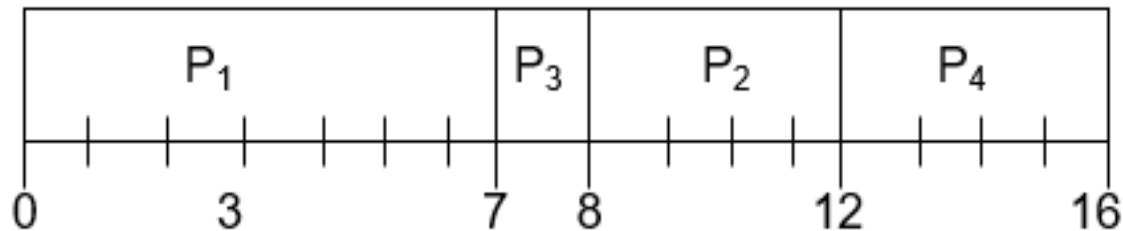


■ Average waiting time = $(3 + 16 + 9 + 0) / 4 = 7$

SJF with arrival Time (non-preemptive)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

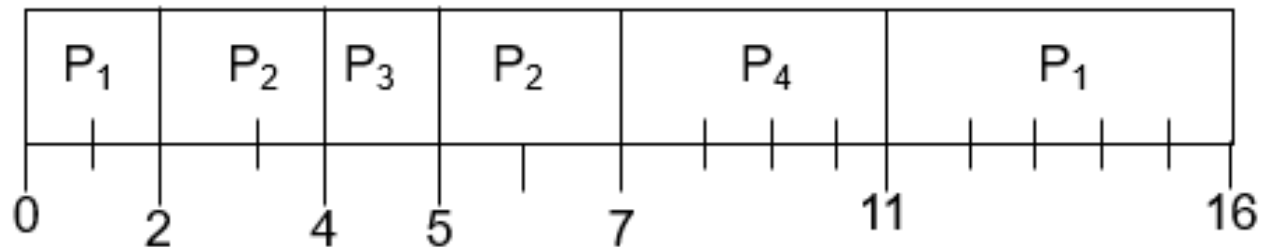


- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

SJF with arrival Time (preemptive)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - Non-preemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process.

Priority Scheduling (Non-preemptive)

Process ID	Priority	Arrival Time	Burst Time
1	2	0	3
2	6	2	5
3	3	1	4
4	5	4	2
5	7	6	9
6	4	5	4
7	10	7	10

Priority Scheduling (Non-preemptive)

P1	P3	P6	P4	P2	P5	P7
0	3	7	11	13	18	27
						37

From the GANTT Chart prepared, we can determine the completion time of every process. The turnaround time, waiting time and response time will be determined.

Turn Around Time = Completion Time - Arrival Time

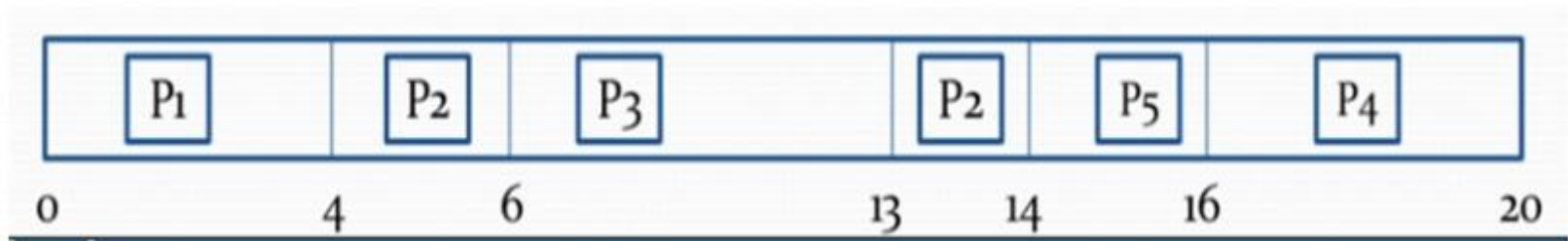
Waiting Time = Turn Around Time - Burst Time

Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time	Response Time
1	2	0	3	3	3	0	0
2	6	2	5	18	16	11	13
3	3	1	4	7	6	2	3
4	5	4	2	13	9	7	11
5	7	6	9	27	21	12	18
6	4	5	4	11	6	2	7
7	10	7	10	37	30	18	27

Avg Waiting Time = $(0+11+2+7+12+2+18)/7 = 52/7$ units

Priority Scheduling (Preemptive)

Process	Priority	Burst time	Arrival time
P1	1	4	0
P2	2	3	0
P3	1	7	6
P4	3	4	11
P5	2	2	12



Priority Scheduling (Preemptive)



Formula:

Waiting Time = Start Time - Arrival Time + Wait Time for Next Burst

Individual Waiting Times:

$$P1 = 0 - 0 = 0$$

$$P2 = 4 - 0 + 7 = 11$$

$$P3 = 6 - 6 = 0$$

$$P4 = 16 - 11 = 5$$

$$P5 = 16 - 14 = 2$$

Average Waiting Time:

$$\text{Average Waiting Time} = \frac{0 + 11 + 0 + 5 + 2}{5} = \frac{18}{5} = 3.6$$

Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum q**), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
- No process waits more than $(n-1)*q$ time units. Timer interrupts every quantum to schedule the next process.

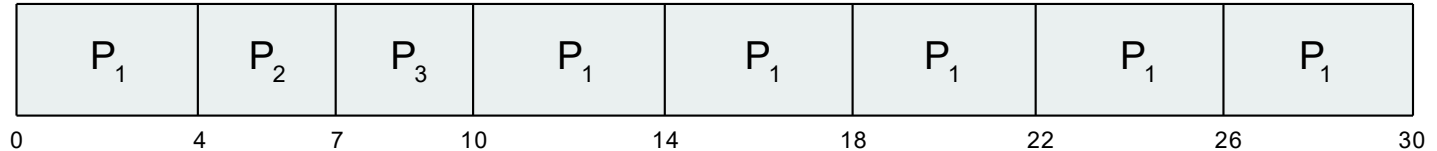
Performance

- If q is large:
 - Round Robin behaves like **First-Come, First-Served (FCFS)**.
 - Less context switching, but poorer responsiveness.
- If q is small:
 - Better responsiveness (true RR behaviour).
 - However, **too many context switches** increase overhead.

Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

■ The Gantt chart is:



■ Let's calculate the average waiting time for this schedule.

P_1 waits for 6 milliseconds ($10 - 4$),

P_2 waits for 4 milliseconds, and

P_3 waits for 7 milliseconds.

■ Thus, the average waiting time is $17/3 = 5.66$ milliseconds.

■ Typically, higher average turnaround than SJF, but better *response*

■ q **should be large compared to context switch time**

- q usually 10 milliseconds to 100 milliseconds,

- Context switch < 10 microseconds

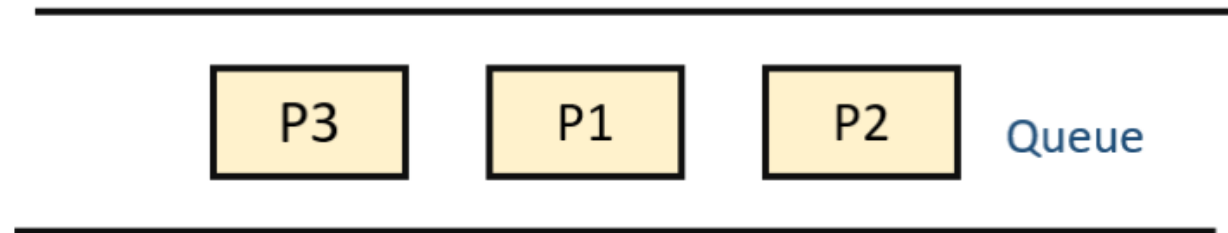
-
- In the RR scheduling algorithm, no process is allocated the CPU for more than 1 time quantum in a row (unless it is the only runnable process).
 - If a process's CPU burst exceeds 1 time quantum, that process is preempted and is put back in the ready queue. The RR scheduling algorithm is thus preemptive.
 - If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units. Each process must wait no longer than $(n - 1) \times q$ time units until its next time quantum.
 - For example, with five processes and a time quantum of 20 milliseconds, each process will get up to 20 milliseconds every 100 milliseconds.

Example of RR with Time Quantum = 2

Example of Round-robin Scheduling

Consider this following three processes

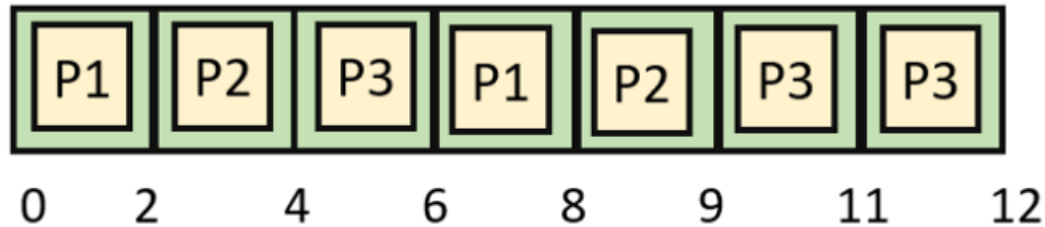
Process Queue	Burst time
P1	4
P2	3
P3	5



Time Slice = 2

Example of RR with Time Quantum = 2

Time Slice = 2



Wait time

$$P1 = 0 + 4 = 4$$

$$P2 = 2 + 4 = 6$$

$$P3 = 4 + 3 = 7$$

Time Quantum and Context Switch Time

